INVENTORY MANAGEMENT SYSTEM

Angular Frontend Specification

Version 1.0 • Angular 18+ • Standalone Components • Signal-based State

1. Project Overview

This document specifies the Angular frontend application for the Inventory Management System (IMS). It defines the project structure, module design, routing, component catalogue, service layer, state management, API integration, and UI/UX guidelines. The frontend consumes the Spring Boot backend REST API documented in the backend specification.

The application targets operations staff who manage physical goods across multiple markets. It must be fast, reliable, and usable on both desktop and tablet form factors.

1.1 Goals

- Provide a complete UI for all backend use cases: items, markets, stock operations, pricing, and reporting
- Use a modern Angular (18+) standalone component architecture with signals
- Enforce strict separation between presentation, state, and API integration layers
- Support real-time or near-real-time stock feedback via polling or WebSocket upgrade path
- Be containerized and served via Nginx in Docker

1.2 Technology Stack

Concern	Technology / Library
Framework	Angular 18+ (standalone components, no NgModules)
Language	TypeScript 5.4+
State Management	Angular Signals + signal-based services (no NgRx for v1)
HTTP Client	Angular HttpClient with typed interceptors
Routing	Angular Router with lazy-loaded routes
Forms	Angular Reactive Forms
UI Component Library	Angular Material 18 (mat-*)
Charts / Data Vis	ng2-charts (Chart.js wrapper)
Styling	SCSS + Angular Material theming
Build Tool	Angular CLI (ng build)
Code Quality	ESLint + Prettier + Husky pre-commit hooks
Containerization	Docker + Nginx (alpine)
API Proxy (dev)	Angular CLI proxy config → http://localhost:8080
Date Handling	date-fns

2. Project Structure

2.1 Repository Layout

The Angular project follows a feature-based folder structure. Each domain feature is self-contained with its own components, services, and routes.

```
ims-frontend/
  src/
    app/
      core/                       # App-wide singletons (auth, interceptors, error
handling)
        interceptors/
          auth.interceptor.ts
          error.interceptor.ts
          loading.interceptor.ts
        services/
          notification.service.ts
          loading.service.ts
        guards/
          auth.guard.ts
      shared/                     # Reusable components, pipes, directives
        components/               # confirm-dialog, status-badge, money-display,
etc.
        pipes/                    # currency-format, date-format, stock-level
        directives/               # loading-spinner, permission
        models/                   # shared TypeScript interfaces
      features/                   # One folder per backend domain
        items/
        markets/
        market-stock/
        transactions/
        reports/
        storage/
      layout/                     # Shell: sidebar, topbar, breadcrumbs
    assets/
    environments/
      environment.ts              # { apiBaseUrl: '/api/v1', production: false }
      environment.prod.ts
  proxy.conf.json                # Dev proxy to backend
  nginx.conf                     # Production Nginx config
  Dockerfile
  docker-compose.yml
```

2.2 Feature Module Anatomy

Every feature folder follows the same internal structure:

```
features/items/
  components/
    item-list/          item-list.component.ts/.html/.scss
    item-detail/        item-detail.component.ts/.html/.scss
    item-form/          item-form.component.ts/.html/.scss
    stock-adjust/       stock-adjust.component.ts/.html/.scss
  services/
    item-api.service.ts        # HttpClient calls to /api/v1/items
    item-state.service.ts      # Signal-based state store
  models/
    item.model.ts              # TypeScript interfaces matching API DTOs
  items.routes.ts              # Lazy route definitions
```

3. Routing Architecture

3.1 Route Tree

All feature routes are lazy-loaded. The shell layout (sidebar + topbar) wraps all authenticated routes via a layout component outlet.

Path	Lazy Feature	Component / View
/	Redirect	Redirects to /dashboard
/dashboard	DashboardFeature	Cross-market summary widget board
/items	ItemsFeature	Item list with search and filters
/items/new	ItemsFeature	Register new item form
/items/:id	ItemsFeature	Item detail — stock, position, history
/items/:id/edit	ItemsFeature	Edit item metadata
/markets	MarketsFeature	Market list with status badges and filters
/markets/new	MarketsFeature	Create new market form
/markets/:id	MarketsFeature	Market detail — items, stock, lifecycle actions
/markets/:id/items/add	MarketStockFeature	Shift items to market
/markets/:id/items/:itemId	MarketStockFeature	Market item detail — price, stock, transactions
/markets/:id/summary	ReportsFeature	Market sales and stock summary
/reports	ReportsFeature	All-markets aggregate report with filters
/storage	StorageFeature	Storage stock levels and positions
/transactions	TransactionsFeature	Global transaction log
**	NotFoundComponent	404 page

3.2 Route Guards

- AuthGuard — redirects to /login if user is unauthenticated (stubbed for v1, wired for future Spring Security integration)
- MarketStatusGuard — prevents navigation to shift/stock routes for CLOSED markets; redirects to market detail with a toast notification
- UnsavedChangesGuard — prompts confirmation dialog if a form has unsaved changes

4. Feature Specifications

4.1 Items Feature

4.1.1 Item List Page

URL: /items — Displays a paginated, searchable table of all registered items.

UI Element	Behaviour / Binding
Search bar	Debounced 300ms text input — filters by name or SKU via GET /items?sku=...
Category filter chip group	Multi-select chips; triggers re-fetch with category param
Items table	Columns: SKU, Name, Category, Default Price, Storage Stock, Position, Actions
Storage stock badge	Green if stock > 10, Amber if 1–10, Red if 0 — uses StockLevelPipe
Actions per row	View button → /items/:id; Edit button → /items/:id/edit
Add Item button (top-right)	Navigates to /items/new
Paginator	mat-paginator; page size options: 10, 25, 50; synced to ?page=&size= query params
Export button	Downloads CSV of current filtered result via client-side generation

4.1.2 Item Detail Page

URL: /items/:id — Full detail of a single item across two tabs.

- Tab 1 — Overview: item card with all metadata fields, default price, storage position map
- Tab 2 — Transactions: paginated table of all STOCK_ADJUSTMENT and SHIFT_TO_MARKET transactions for this item
- Adjust Stock FAB (floating action button) — opens stock-adjust side panel
- Edit button in header — navigates to /items/:id/edit
- Markets using this item — chip list showing markets where the item is allocated, each links to /markets/:id/items/:itemId

4.1.3 Item Form (Register / Edit)

URL: /items/new or /items/:id/edit — Reactive form with full validation.

Field	Control Type	Validation
SKU	mat-input text	Required; pattern [A-Z0-9-]{3,20}; async unique check via API (edit: skip if unchanged)

Field	Control Type	Validation
Name	mat-input text	Required; min 2, max 100 chars
Description	mat-textarea	Optional; max 500 chars
Category	mat-select (with option to type new)	Required
Default Price (amount)	mat-input number	Required; min 0; max 2 decimal places
Default Price (currency)	mat-select ISO codes	Required; default SEK
Storage Zone	mat-input text	Required
Storage Shelf	mat-input text	Required
Storage Row	mat-input number (integer)	Required; min 1
Storage Column	mat-input number (integer)	Required; min 1
Initial Stock (register only)	mat-input number	Required on create; min 0

- On submit: POST /items (register) or PUT /items/:id (update)
- On success: navigate to /items/:id with success snackbar
- On error: display inline field-level errors from API 400 response

4.1.4 Stock Adjust Side Panel

Triggered from item detail. Slides in as a mat-drawer from the right.

- Delta input (positive or negative integer) with ± toggle buttons
- Note textarea (optional)
- Preview shows: Current stock → New stock (calculated live using signals)
- Disabled if new stock would be negative — shows inline warning
- Submit calls PATCH /items/:id/stock; on success refreshes item signal and appends transaction to tab 2

4.2 Markets Feature

4.2.1 Market List Page

URL: /markets — Displays all markets with lifecycle status and quick actions.

UI Element	Behaviour / Binding
Status filter tabs	All / Scheduled / Open / Closed — maps to MarketStatus enum
Place filter dropdown	Distinct places from API response; triggers re-fetch

UI Element	Behaviour / Binding
Date range picker	From / To date range; filters markets by openDate–closeDate overlap
Markets grid / table toggle	Grid view shows cards; Table view shows dense rows; preference saved to localStorage
Market card	Shows: name, place, status badge, open/close dates, item count, quick-action buttons
Open action (Scheduled only)	Confirmation dialog → POST /markets/:id/open
Close action (Open only)	Confirmation dialog with reconciliation warning → POST /markets/:id/close
View button	Navigates to /markets/:id
Create Market FAB	Navigates to /markets/new

4.2.2 Market Detail Page

URL: /markets/:id — Central hub for a market's lifecycle, items, and stock management.

- Market header card: name, place, dates, status badge, open/close action buttons (context-aware)
- Tab 1 — Items at Market: table of all MarketItems with columns: Item Name, SKU, Allocated Stock, Current Stock, Market Price, Actions
- Tab 2 — Summary: embedded market summary widget (same as /markets/:id/summary)
- Tab 3 — Transactions: paginated log of all transactions for this market
- Shift Items button (visible when OPEN): navigates to /markets/:id/items/add

> **Note:** *Open / Close buttons perform a confirmation dialog before calling the API. Close dialog warns that remaining stock will be returned to storage.*

4.2.3 Create Market Form

URL: /markets/new

Field	Control Type	Validation
Market Name	mat-input text	Required; min 3, max 100 chars
Place / Venue	mat-input text with autocomplete from past places	Required; min 2, max 100 chars
Open Date	mat-datepicker	Required; must not be in the past
Close Date	mat-datepicker	Required; must be after Open Date

- POST /markets on submit; navigates to /markets/:id on success

4.3 Market Stock Feature

4.3.1 Shift Items to Market Page

URL: /markets/:id/items/add — Multi-step wizard to allocate storage items to a market.

- Step 1 — Item Selection: searchable list of all items with available storage stock > 0; multi-select checkboxes
- Step 2 — Quantities: for each selected item, show a stepper input (min: 1, max: current storage stock); live remaining-stock preview
- Step 3 — Review: read-only summary table of items and quantities before submission
- Submit: fires POST /markets/:id/items for each item sequentially (or batch if API supports it); shows progress indicator
- On success: navigate back to /markets/:id (Tab 1) with success snackbar

4.3.2 Market Item Detail Page

URL: /markets/:id/items/:itemId — Manage a single item's stock and price at a market.

UI Section	Content
Stock Card	Current stock at market, allocated stock, total sold (derived). Increment and Decrement buttons.
Increment Stock dialog	Input quantity; calls PATCH /markets/:id/items/:itemId/increment
Decrement Stock dialog	Input quantity with optional note (e.g. SALE, DAMAGED); calls PATCH /markets/:id/items/:itemId/decrement; blocked if quantity > currentStock
Price Card	Current market price with Edit button. Opens inline edit form → PUT /markets/:id/items/:itemId/price
Transactions table	Paginated transactions for this item at this market; shows type badge, delta, stock before/after, note, timestamp

> **Note:** *Increment and Decrement dialogs show live stock preview and disable confirm button if the result would be negative.*

4.4 Reports Feature

4.4.1 Market Summary Page

URL: /markets/:id/summary — Visual sales and stock report for a single market.

- Summary KPI row: Total Items, Total Units Sold, Total Revenue (sum of decrements x price), Current Stock Total
- Revenue bar chart (ng2-charts): X-axis = item name, Y-axis = revenue per item

- Stock status doughnut chart: proportion of Sold vs Remaining vs Returned
- Transactions timeline: sparkline of transaction volume per day
- Per-item breakdown table: columns: Item, Allocated, Sold, Remaining, Revenue, Market Price
- Export as CSV button

4.4.2 All-Markets Report Page

URL: /reports — Aggregate view across all markets with filtering.

Filter / Control	Behaviour
Market multi-select	Filter by one or more market IDs
Place filter	Filter by venue/place
Status filter	SCHEDULED / OPEN / CLOSED / All
Date range picker	Narrow to markets within a date window
Apply button	Calls GET /markets/summary with constructed query params

- Summary KPI row (totals across filtered markets): Total Markets, Total Revenue, Total Units Sold, Total Items
- Market comparison bar chart: revenue per market
- Place comparison chart: total revenue grouped by place
- Markets summary table: one row per market with all KPIs; sortable columns
- Export report as CSV

4.5 Storage Feature

4.5.1 Storage Overview Page

URL: /storage — Full view of all items in storage with their positions and stock levels.

- Zone grouping: items grouped by storage zone; each zone shown as an expandable panel
- Search / filter: filter by item name, SKU, or shelf
- Storage grid: visual grid of shelves and rows (colour-coded by stock level)
- Per-item row: Shelf, Row, Column, Current Stock, Last Adjusted, Quick Adjust button
- Quick Adjust: opens same stock-adjust side panel as item detail
- Low stock alert banner: if any item has stock = 0, show a dismissible warning with links

4.6 Transactions Feature

URL: /transactions — Global audit log of all transactions across all markets and storage.

Column	Description
Timestamp	Formatted date + time; sortable
Type badge	Colour-coded: SHIFT_TO_MARKET (blue), SALE (green), RETURN (orange), STOCK_ADJUSTMENT (gray), MARKET_CLOSE_RECONCILE (purple)
Market	Market name; links to /markets/:id
Item	Item name + SKU; links to /items/:id
Delta	Signed number: +N or -N in green/red
Stock Before / After	Shows movement context
Note	Operator note (truncated; tooltip on hover)
Created By	User or system identifier

- Filters: date range, transaction type, market, item
- Pagination: server-side with mat-paginator
- Export CSV of current filtered view

5. Shared Component Library

Component / Pipe / Directive	Location	Purpose
StatusBadgeComponent	shared/components	Renders a MarketStatus or TransactionType as a colour-coded mat-chip
StockLevelComponent	shared/components	Displays stock quantity with traffic-light colour (red/amber/green) and optional threshold config
MoneyDisplayComponent	shared/components	Formats Money value object (amount + currency) using Intl.NumberFormat
ConfirmDialogComponent	shared/components	Generic confirmation mat-dialog with configurable title, message, and action labels; returns boolean Observable
PageHeaderComponent	shared/components	Standard page title + breadcrumb + optional action buttons slot
EmptyStateComponent	shared/components	Illustrated empty-state placeholder with configurable title and call-to-action
LoadingOverlayComponent	shared/components	Full-screen or contained spinner overlay; controlled by LoadingService
CurrencyFormatPipe	shared/pipes	Transforms BigDecimal + currency string to locale-formatted currency string
DateFormatPipe	shared/pipes	Wraps date-fns format; outputs human-readable relative or absolute dates
StockLevelPipe	shared/pipes	Maps integer stock to 'low' | 'medium' | 'ok' status string for CSS classes
TransactionTypePipe	shared/pipes	Maps TransactionType enum value to display label and colour token
LoadingDirective	shared/directives	Structural directive *loading that shows a spinner while an Observable is pending
PermissionDirective	shared/directives	*hasPermission — stub for future role-based visibility; hides element if permission not met

6. State Management

6.1 Signal-Based Services

Each feature has a dedicated state service using Angular Signals. There is no global store in v1. State services are provided at the root level for cross-component sharing within a feature.

State Service	Signals Exposed	Mutations
ItemStateService	items: Signal<Item[]>, selectedItem: Signal<Item|null>, loading: Signal<boolean>, error: Signal<string|null>	loadItems(), loadItem(id), registerItem(cmd), updateItem(cmd), adjustStock(cmd)
MarketStateService	markets: Signal<Market[]>, selectedMarket: Signal<Market|null>, loading, error	loadMarkets(filters), loadMarket(id), createMarket(cmd), openMarket(id), closeMarket(id)
MarketStockStateService	marketItems: Signal<MarketItem[]>, loading, error	loadMarketItems(marketId), shiftItem(cmd), increment(cmd), decrement(cmd), setPrice(cmd)
TransactionStateService	transactions: Signal<Page<Transaction>>, loading, error	loadTransactions(filters), loadForMarket(id), loadForItem(id)
ReportStateService	marketSummary: Signal<MarketSummary|null>, allSummary: Signal<AllMarketsSummary|null>	loadMarketSummary(id), loadAllSummary(filters)
StorageStateService	storageItems: Signal<StorageItem[]>, loading, error	loadStorageItems(), adjustStock(cmd)

6.2 Computed Signals

Derived state is expressed as computed() signals to avoid redundant recalculation:

- totalStorageStock = computed(() => storageItems().reduce((sum, i) => sum + i.stock, 0))
- openMarketsCount = computed(() => markets().filter(m => m.status === 'OPEN').length)
- lowStockItems = computed(() => items().filter(i => i.totalStorageStock <= 5))
- marketRevenue = computed(() => marketSummary()?.totalRevenue ?? 0)

6.3 Loading & Error Strategy

- Every state service exposes a loading signal set to true before HTTP calls and false after
- Errors are set on the error signal and displayed via a shared ErrorBannerComponent

- Global HTTP errors (500, network) are caught by ErrorInterceptor and dispatched to NotificationService (toast)
- 404 errors from detail pages navigate to a NotFoundComponent

7. API Integration Layer

7.1 API Service Pattern

Each feature has an *-api.service.ts that wraps Angular HttpClient. These services are the only place that knows about HTTP paths. State services call API services and update signals.

API Service	Methods (all return Observable<T>)
ItemApiService	getItems(params), getItem(id), registerItem(cmd), updateItem(id, cmd), adjustStock(id, delta, note), getItemTransactions(id, params)
MarketApiService	getMarkets(params), getMarket(id), createMarket(cmd), openMarket(id), closeMarket(id), getMarketSummary(id), getAllMarketsSummary(params)
MarketStockApiService	getMarketItems(marketId), shiftItem(marketId, cmd), getMarketItem(marketId, itemId), incrementStock(marketId, itemId, qty), decrementStock(marketId, itemId, qty, note), setMarketItemPrice(marketId, itemId, price)
TransactionApiService	getTransactions(params), getMarketTransactions(marketId, params), getItemTransactions(itemId, params), getMarketItemTransactions(marketId, itemId, params)
StorageApiService	getStorageSummary()

7.2 HTTP Interceptors

Interceptor	Responsibility
AuthInterceptor	Attaches Authorization header (Bearer token stub for v1; reads from localStorage)
LoadingInterceptor	Increments / decrements LoadingService counter; shows global loading bar
ErrorInterceptor	Catches 4xx/5xx responses; formats error messages; routes 401 to login, 403 to forbidden page, 500 to toast
CorrelationIdInterceptor	Attaches X-Correlation-ID header (UUID per request) to aid backend log tracing
CacheInterceptor	Caches GET /items and GET /markets responses for 30s with ETag support (opt-in via request context)

7.3 TypeScript Models (matching backend DTOs)

```
// Money value object
export interface Money { amount: number; currency: string; }
```

```
// StoragePosition
export interface StoragePosition { zone: string; shelf: string; row: number;
column: number; }

// Item
export interface Item {
  id: string; sku: string; name: string; description?: string;
  category: string; defaultPrice: Money; storagePosition: StoragePosition;
  totalStorageStock: number; createdAt: string; updatedAt: string;
}

// Market
export interface Market {
  id: string; name: string; place: string;
  openDate: string; closeDate: string;
  status: 'SCHEDULED' | 'OPEN' | 'CLOSED';
  createdAt: string;
}

// MarketItem
export interface MarketItem {
  id: string; marketId: string; itemId: string;
  allocatedStock: number; currentStock: number; marketPrice: Money;
}

// Transaction
export interface Transaction {
  id: string; marketId?: string; itemId: string;
  type: TransactionType; quantityDelta: number;
  stockBefore: number; stockAfter: number;
  note?: string; occurredAt: string; createdBy: string;
}

export type TransactionType = 'SHIFT_TO_MARKET' | 'SALE' | 'RETURN' |
'STOCK_ADJUSTMENT' | 'MARKET_CLOSE_RECONCILE';

// Pagination wrapper
export interface Page<T> { content: T[]; totalElements: number; totalPages: number;
number: number; size: number; }
```

8. UI/UX Design Guidelines

8.1 Layout Shell

- Fixed left sidebar (240px) with navigation links grouped by domain: Items, Markets, Storage, Reports, Transactions
- Top bar: application title, breadcrumb, user avatar (stub), notifications bell
- Content area: scrollable, max-width 1400px centred
- Responsive breakpoints: sidebar collapses to icons-only at < 1280px, drawer at < 768px

8.2 Colour System

Token	Hex	Usage
Primary	#1B3A6B	App branding, headers, primary buttons
Accent	#C0392B	Angular Material accent, destructive actions
Success	#27AE60	Stock OK, positive deltas, success toasts
Warning	#F39C12	Low stock, amber badges
Error / Danger	#E74C3C	Zero stock, error states, delete actions
Surface	#F8F9FA	Page backgrounds
On-Surface	#1A1A1A	Primary text
Muted	#6C757D	Secondary text, helper labels

8.3 Stock Level Colour Convention

Applied consistently across all stock displays via StockLevelComponent and StockLevelPipe:

- Stock = 0: Red background, white text badge — CRITICAL
- Stock 1–5: Amber background, dark text badge — LOW
- Stock 6–20: Standard text, no badge
- > 20: Green text indicator — WELL STOCKED

> **Note:** *Thresholds are configurable per-deployment via environment config.*

8.4 Form Conventions

- All forms use Reactive Forms with typed FormGroup<T>
- Validation errors shown below each field using mat-error; messages written in plain language
- Forms auto-save draft to sessionStorage on blur (register/edit forms only); restored on page reload
- Submit buttons disabled while loading; show mat-spinner inside button
- Cancel / Back always navigates up one level in the route tree

8.5 Notification System

- Success actions: green snackbar bottom-right, auto-dismiss 3s
- Warnings: amber snackbar, auto-dismiss 5s
- Errors: red snackbar, manual dismiss with View Details link opening a dialog
- Confirmation dialogs: used for all destructive or irreversible actions (open market, close market, adjust stock)

9. Containerization

9.1 Dockerfile

Multi-stage build — Stage 1 builds the Angular app, Stage 2 serves it via Nginx:

```
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package*.json ./
RUN npm ci
COPY . .
RUN npm run build -- --configuration production

# Stage 2: Serve
FROM nginx:1.27-alpine
COPY --from=builder /app/dist/ims-frontend/browser /usr/share/nginx/html
COPY nginx.conf /etc/nginx/conf.d/default.conf
EXPOSE 80
```

9.2 Nginx Configuration

Nginx must serve the Angular SPA with HTML5 routing (try_files) and proxy /api/v1 to the Spring Boot backend:

```
server {
  listen 80;
  root /usr/share/nginx/html;
  index index.html;

  location /api/ {
    proxy_pass http://ims-app:8080;
    proxy_set_header Host $host;
  }

  location / {
    try_files $uri $uri/ /index.html;
  }
}
```

9.3 Docker Compose Addition

Add the following service to the backend docker-compose.yml:

```
  ims-frontend:
    build: ./ims-frontend
    ports:
      - "4200:80"
    depends_on:
      - ims-app
    environment:
      - API_BASE_URL=http://ims-app:8080
```

10. Code Quality & Conventions

10.1 TypeScript Configuration

- strict: true — all strict type-checking flags enabled
- noImplicitAny: true
- strictNullChecks: true
- Target: ES2022

10.2 Linting & Formatting

- ESLint: @angular-eslint ruleset + no-explicit-any, prefer-const, no-unused-vars
- Prettier: enforced via eslint-plugin-prettier; line length 100, single quotes, trailing commas
- Husky pre-commit hook: runs eslint --fix and prettier --write on staged files
- lint-staged configured to run on *.ts, *.html, *.scss only

10.3 Naming Conventions

Element	Convention	Example
Components	kebab-case selector; PascalCase class	app-item-list, ItemListComponent
Services	PascalCase; -Service suffix	ItemApiService, MarketStateService
Interfaces / Models	PascalCase; no I prefix	Item, Market, MarketSummary
Signals	camelCase; no special suffix	items, selectedMarket, loading
Route files	feature-name.routes.ts	items.routes.ts
SCSS files	component scoped; BEM-like naming	.item-list__header--highlighted
Environment keys	SCREAMING_SNAKE_CASE	API_BASE_URL, STOCK_LOW_THRESHOLD

11. Dashboard Page

URL: /dashboard — Landing page after login. Provides an at-a-glance operational overview.

Widget	Data Source	Detail
Open Markets KPI card	GET /markets?status=OPEN	Count of currently open markets; links to /markets?status=OPEN
Low Stock Alerts card	GET /items (client computed)	Items with storage stock ≤ 5; expandable list with quick-adjust link
Today's Transactions card	GET /transactions?from=today	Count of transactions today; links to /transactions
Total Inventory Value card	GET /items (client computed)	Sum of (defaultPrice.amount × totalStorageStock) across all items
Revenue by Market chart	GET /markets/summary	Horizontal bar chart; top 5 markets by revenue
Market Status distribution	GET /markets	Doughnut chart: SCHEDULED / OPEN / CLOSED counts
Recent Transactions table	GET /transactions?size=10&sort=occurredAt,desc	Last 10 transactions with type badge, item, delta
Markets closing soon	GET /markets?status=OPEN	Markets whose closeDate is within 7 days; shown as warning cards

12. Angular CLI Bootstrap

12.1 Project Generation Command

Run the following Angular CLI command to scaffold the project:

```
ng new ims-frontend \
  --routing=true \
  --style=scss \
  --strict \
  --standalone \
  --ssr=false
```

12.2 Required npm Packages

Package	Version	Purpose
@angular/material	^18.0.0	UI component library
@angular/cdk	^18.0.0	Angular CDK (required by Material)
ng2-charts	^6.0.0	Chart.js Angular wrapper
chart.js	^4.0.0	Chart rendering engine
date-fns	^3.0.0	Date manipulation and formatting
@angular-eslint/schematics	^18.0.0	ESLint Angular ruleset
eslint-config-prettier	^9.0.0	ESLint + Prettier integration
prettier	^3.0.0	Code formatter
husky	^9.0.0	Git hooks
lint-staged	^15.0.0	Run linters on staged files

13. Future Considerations (Out of Scope v1)

- Authentication UI: Login page, JWT storage, token refresh flow, role-based UI rendering
- Real-time stock updates: WebSocket or SSE subscription to backend events via Kafka consumer endpoint
- Progressive Web App (PWA): service worker for offline capability and push notifications
- Unit & integration tests: Jest + Angular Testing Library (to be specified separately)
- E2E tests: Playwright or Cypress
- Internationalisation (i18n): Angular i18n or ngx-translate for Swedish/English
- Dark mode: CSS custom property theming toggle
- NgRx migration: if state complexity grows beyond signal-based services
- Print / PDF export of summary reports
- Barcode / QR scanner integration for item lookup on mobile